

Yabloko (Яблоко)

Maksym Mykhailych, Bartolmay Can, Michael Kurz,
Schayan Goudarzi, Tim Bornemann

Software Engineering – Dokumentation

25. Juni 2026

Inhaltsverzeichnis

Kurzfassung	3
1 Einleitung	3
1.1 Motivation und Problemstellung	3
1.2 Zielsetzung der Arbeit	3
1.3 Aufbau der Dokumentation	3
2 Grundlagen	4
2.1 Einführung in Software Engineering	4
2.2 Verwendete Methoden / Modelle	4
2.3 Technologische Grundlagen	4
3 Anforderungsanalyse	5
3.1 Stakeholderanalyse	5
3.2 Funktionale Anforderungen	5
3.3 Nicht-funktionale Anforderungen	6
3.4 User Stories	7
3.5 Priorisierung der Anforderungen	8
4 Use-Case-Modellierung	9
4.1 Use Case: Spiel starten	9
4.2 Use Case: Gegnerkontakt	12
5 Systementwurf (Design)	16
5.1 Architekturüberblick	16
5.2 Komponenten- und Moduldesign	16
5.3 UML-Diagramme	17
5.3.1 Klassendiagramm	17
5.3.2 Zustandsdiagramm	18
5.4 Zentrale Spielobjekte	18
5.4.1 Gegner	18
5.4.2 Level Objects	19
5.4.3 Pickups	19
6 Implementierung	19
6.1 Verwendete Technologien und Tools	19
6.2 Aufbau der Codebasis	20
6.3 Zentrale Implementierungsaspekte	21
6.3.1 Spielersteuerung und Bewegung	21
6.3.2 Gegner	22
6.3.3 Kampfsystem	22
6.3.4 Power-Ups und Sammelobjekte	22
6.3.5 Fortschritt, Leben und Persistenz	22
6.3.6 Audio- und Effektsystem	22
6.3.7 Benutzeroberfläche	22
6.3.8 Verwendete Entwurfsmuster	23
6.4 Herausforderungen während der Umsetzung	23
7 Qualitätssicherung und Test	23
7.1 Teststrategie	23
7.2 Unit-Testplan mit Äquivalenzklassen	23

7.3	Testfallermittlung mittels Äquivalenzklassen	24
7.4	Erweiterbarkeit des Testkonzepts	24
7.5	Testfälle und Durchführung	24
7.6	Testergebnisse und Analyse	26
7.7	Automatisierte Testausführung mittels GitHub Actions	26
8	Projektmanagement	26
8.1	Vorgehensmodell	26
8.2	Zeit- und Meilensteinplanung	27
8.3	Rollenverteilung im Team	28
8.4	Risikomanagement	28
9	Evaluation	29
9.1	Bewertung der Zielerreichung	29
9.2	Vergleich mit Anforderungen	29
9.3	Diskussion der Ergebnisse	29
10	Fazit und Ausblick	29
10.1	Zusammenfassung der Ergebnisse	29
10.2	Kritische Reflexion (Lessons Learned)	30
10.3	Weiterentwicklungsmöglichkeiten	30
11	Anhang	30
11.1	Wichtige Links	30

Kurzfassung

Anmerkung: Einige Diagramme sind in diesem Dokument aufgrund ihrer Größe schwer lesbar. Wir bitten darum, diese bei Bedarf als separate Dateien herunterzuladen.

Dieses Projekt umfasst die Entwicklung eines 2D-Mario-ähnlichen Spiels, das mit der Game Engine Godot (C#, Version 4.6.2) realisiert wurde. Der Spieler steuert eine Figur über vordefinierte Level, weicht Hindernissen aus, bekämpft Gegner und sammelt Belohnungen.

Für die Audio-Gestaltung kamen FL Studio 2025 sowie das VST-Plugin Serum zum Einsatz, um Musik und Soundeffekte zu komponieren und anschließend als `.wav`-Dateien (Effekte) und `.ogg`-Dateien (Musik) ins Spiel zu integrieren. Die Grafiken wurden mit Pixelorama erstellt, ergänzt durch frei verfügbare Asset-Packs. Optisch orientiert sich das Spiel am 16-Bit-Stil klassischer Retro-Spiele.

Die Projektarbeit basiert auf einem an Scrum angelehnten Vorgehen, das um eigene Methoden ergänzt wurde und einen iterativen Entwicklungsprozess mit Sprints und regelmäßigen Reviews ermöglichte. Zentrale Spielmechaniken wie Spielerbewegung, Kollisionserkennung und Levelaufbau sind vollständig implementiert.

Die Anforderungen wurden als User Stories erfasst und in der Gruppe nach Aufwand und Wichtigkeit (Werte: 0, 1, 2, 3, 5, 8, 10) bewertet und anschließend priorisiert.

1 Einleitung

1.1 Motivation und Problemstellung

Im Rahmen des Moduls *Software Engineering 1* wurde ein praxisorientiertes Projekt durchgeführt, dessen Ziel die Entwicklung eines einfachen 2D-Plattformspiels ist. Das Spiel orientiert sich konzeptionell an klassischen Jump-and-Run-Spielen, bei denen eine Spielfigur durch verschiedene Level navigiert, Hindernisse überwindet und Objekte wie Münzen sammelt.

Die Motivation liegt darin, theoretische Inhalte aus dem Software Engineering praktisch anzuwenden. Insbesondere sollen grundlegende Konzepte wie Anforderungsanalyse, Systementwurf, Implementierung und Testen im Rahmen eines realistischen Projekts vertieft werden.

Eine zentrale Herausforderung besteht darin, die einzelnen Komponenten eines Spiels – wie Spiellogik, Leveldesign, Benutzeroberfläche und Sound – sinnvoll zu strukturieren und im Team zu koordinieren. Zudem erfordert die Entwicklung die korrekte Umsetzung von Mechaniken wie Bewegung, Kollisionserkennung und Interaktion mit der Spielumgebung.

1.2 Zielsetzung der Arbeit

Ziel dieses Projekts ist die Entwicklung eines funktionsfähigen Prototyps eines Plattformspiels mit grundlegenden Spielelementen. Dazu zählen unter anderem die Bewegung der Spielfigur (z. B. Springen und Bewegen), das Sammeln von Objekten sowie das Durchlaufen mehrerer Level mit unterschiedlichen Schwierigkeitsgraden.

Ein weiterer wichtiger Schwerpunkt liegt auf der Anwendung strukturierter Vorgehensweisen aus dem Software Engineering. Das Projekt dient somit nicht nur der Entwicklung eines Spiels, sondern vor allem dem Verständnis des gesamten Softwareentwicklungsprozesses – von der Planung über die Umsetzung bis hin zur Qualitätssicherung.

1.3 Aufbau der Dokumentation

Die vorliegende Dokumentation ist wie folgt aufgebaut: Zunächst werden in Kapitel 2 die theoretischen Grundlagen sowie die verwendeten Methoden und Technologien erläutert. Kapitel 3 behandelt

die Anforderungsanalyse. Dabei werden die relevanten Stakeholder, funktionale und nicht-funktionale Anforderungen, User Stories sowie die Priorisierung der Anforderungen beschrieben.

Kapitel 4 widmet sich der Use-Case-Modellierung. Dort werden zwei zentrale Anwendungsfälle des Spiels beschrieben und mithilfe von Aktivitäts- und Sequenzdiagrammen dargestellt.

In Kapitel 5 wird der Systementwurf erläutert. Dazu gehören der Architekturüberblick, das Komponenten- und Moduldesign sowie die verwendeten UML-Diagramme. Kapitel 6 beschreibt die Implementierung des Spiels, einschließlich der verwendeten Technologien, der Projektstruktur, zentraler Implementierungsaspekte und aufgetretener Herausforderungen.

Kapitel 7 behandelt die Qualitätssicherung und die durchgeführten Tests. Neben der Teststrategie werden Unit-Tests, Äquivalenzklassen, manuelle Testfälle, Testergebnisse sowie die automatisierte Testausführung über GitHub Actions beschrieben.

Anschließend werden in Kapitel 8 das Projektmanagement, das Vorgehensmodell, die Zeit- und Meilensteinplanung, die Rollenverteilung sowie das Risikomanagement dargestellt. Kapitel 9 enthält die Evaluation der Projektergebnisse. Abschließend fasst Kapitel 10 die wichtigsten Ergebnisse zusammen, reflektiert die gewonnenen Erfahrungen und gibt einen Ausblick auf mögliche Weiterentwicklungen.

2 Grundlagen

2.1 Einführung in Software Engineering

Software Engineering umfasst die systematische Entwicklung und Wartung von Software unter Anwendung strukturierter Methoden und Werkzeuge. Ziel ist es, Software effizient, nachvollziehbar und in hoher Qualität zu entwickeln.

Der Softwareentwicklungsprozess wird dabei in mehrere Phasen unterteilt, darunter Anforderungsanalyse, Entwurf, Implementierung und Testen. Diese Struktur hilft, komplexe Projekte zu planen und umzusetzen.

Im Rahmen dieses Projekts werden diese Prinzipien angewendet, um ein 2D-Plattformspiel zu entwickeln und gleichzeitig praktische Erfahrungen im Umgang mit Softwareentwicklungsprozessen zu sammeln.

2.2 Verwendete Methoden / Modelle

Für die Umsetzung des Projekts wurde ein iteratives, an agile Methoden angelehntes Vorgehen gewählt. Zu Beginn wurden umfangreiche User Stories erstellt, um die Anforderungen strukturiert zu erfassen.

Die Entwicklung erfolgte in mehreren Sprints. In jedem Sprint wurden die aktuell wichtigsten User Stories umgesetzt, wobei darauf geachtet wurde, dass das Spiel nach jedem Sprint spielbar bleibt.

Aufgrund des kreativen Charakters der Spieleentwicklung wurden die Sprints flexibel gestaltet und nicht strikt vorgegeben. Dies ermöglichte es, auf neue Ideen und Änderungen während der Entwicklung einzugehen.

Zur Abstimmung im Team fanden regelmäßige, wöchentliche Meetings statt. Diese dienten dazu, den aktuellen Fortschritt zu besprechen, Probleme zu identifizieren und die nächsten Schritte zu planen.

2.3 Technologische Grundlagen

Die Entwicklung des Spiels erfolgt mit der Godot Engine 4.6.2, einer leistungsfähigen Open-Source-Game-Engine, die sich besonders für die Entwicklung von 2D-Spielen eignet.

Als Programmiersprache wird C# verwendet, um die Spiellogik und Interaktionen umzusetzen. Die Engine bietet hierfür eine integrierte Entwicklungsumgebung sowie eine strukturierte Szenen- und Node-Architektur, die eine klare Organisation des Projekts ermöglicht.

Zur Versionskontrolle wurde Git in Kombination mit GitHub eingesetzt, was eine parallele Entwicklung im Team ermöglichte. Für die grafische Gestaltung wird das Tool Pixelorama verwendet. Zusätzlich kommen verschiedene frei verfügbare Asset-Packs zum Einsatz. Ziel ist ein Spiel im 16-Bit-Stil, das sich optisch an klassischen Retro-Spielen orientiert.

Die Audioinhalte werden mit FL Studio 2025 sowie dem VST-Plugin Serum und anderen Plugins erstellt. Damit werden sowohl Hintergrundmusik als auch Soundeffekte produziert und anschließend als .wav-Dateien für Effekte und .ogg-Dateien für Musik in das Spiel integriert.

Für unser Vorgehen, das an Scrum angelehnt ist, haben wir das Tool Taiga benutzt, in dem wir ein Projekt erstellt haben und einen gemeinsamen Sprint wöchentlich organisiert haben.

Das entwickelte Spiel ist ein 2D-Singleplayer-Plattformspiel mit levelbasierter Struktur. Zu den implementierten Funktionen gehören unter anderem die Bewegung der Spielfigur, das Springen, das Sammeln von Münzen sowie das Auftreten von Gegnern. Zusätzlich verfügt das Spiel über ein Punktesystem sowie ein Lebenssystem, das bei Verlust aller Leben zum Spielende führt.

3 Anforderungsanalyse

3.1 Stakeholderanalyse

Im Rahmen des Projekts wurden die relevanten Stakeholder identifiziert und hinsichtlich ihrer Interessen analysiert.

Stakeholder	Interesse am System
Spieler	Intuitive Bedienung, stabiles Gameplay und hoher Spielspaß
Projektteam	Erfolgreiche Umsetzung der Anforderungen sowie Erweiterbarkeit des Systems
Dozent	Bewertung der technischen Umsetzung, Dokumentation und Methodenanwendung
Testpersonen / Mitstudierende	Funktionierende Spielmechaniken sowie konstruktives Feedback

Tabelle 1: Stakeholderanalyse

Da das Projekt im Rahmen einer Lehrveranstaltung entstand, existieren keine externen Auftraggeber. Die Anforderungen wurden durch das Projektteam definiert und kontinuierlich verfeinert.

3.2 Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben die Funktionen, die das System bereitstellen muss.

Spielfigur

- Das System muss die horizontale Bewegung der Spielfigur nach links und rechts ermöglichen.
- Das System muss Sprünge sowie einen Doppelsprung unterstützen.
- Das System muss das Ducken der Spielfigur ermöglichen.
- Das System muss Angriffe auf Gegner unterstützen.

Gegner und Kollisionen

- Das System muss Gegner innerhalb der Level bereitstellen.
- Das System muss Kollisionen zwischen Spieler und Gegner erkennen.
- Das System muss auf Kollisionen entsprechend reagieren (Lebensverlust, Schrumpfen oder Eliminierung des Gegners).

Fortschrittssystem

- Das System muss ein Punktesystem bereitstellen.
- Das System muss das Sammeln von Münzen ermöglichen.
- Das System muss Highscores dauerhaft speichern.
- Das System muss den Spielfortschritt innerhalb eines Levels anzeigen.

Levelsystem

- Das System muss mehrere Level bereitstellen.
- Das System muss das erfolgreiche Abschließen eines Levels erkennen.
- Das System muss Checkpoints zur Wiederaufnahme des Spiels nach einem Tod unterstützen.

Benutzeroberfläche

- Das System muss ein Hauptmenü bereitstellen.
- Das System muss ein Pausenmenü bereitstellen.
- Das System muss einen Game-Over-Bildschirm anzeigen.
- Das System muss einen Levelabschluss-Bildschirm anzeigen.
- Das System muss die Anzeige von Punktestand, verbleibenden Leben und Spielzeit ermöglichen.

Audio

- Das System muss Hintergrundmusik unterstützen.
- Das System muss Soundeffekte für Spielereignisse bereitstellen.
- Das System muss die Anpassung der Lautstärke ermöglichen.

3.3 Nicht-funktionale Anforderungen

Neben den funktionalen Anforderungen wurden verschiedene Qualitätsanforderungen definiert.

Performance

- Das Spiel soll auf handelsüblichen Laptops flüssig ausführbar sein.
- Die Framerate soll während des Spielbetriebs stabil bleiben.

Benutzbarkeit

- Die Benutzeroberfläche soll intuitiv und verständlich gestaltet sein.
- Die Steuerung soll individuell konfigurierbar sein.

Wartbarkeit

- Der Quellcode soll modular aufgebaut sein.
- Die Spiellogik soll von Benutzeroberflächen und Hilfssystemen getrennt werden.
- Die Ordnerstruktur soll eine einfache Erweiterung des Projekts ermöglichen.

Zuverlässigkeit

- Das Spiel soll ohne Abstürze oder Datenverlust ausführbar sein.
- Highscores und Benutzereinstellungen sollen dauerhaft gespeichert werden.

Erscheinungsbild

- Das Spiel soll einen einheitlichen 16-Bit-Retro-Stil verwenden.
- Grafiken und Benutzeroberflächen sollen ein konsistentes Design aufweisen.

3.4 User Stories

Zur Erhebung und Priorisierung der Anforderungen wurden User Stories verwendet. Diese beschreiben Anforderungen aus Sicht der späteren Benutzer sowie der Entwickler.

Menü und Benutzeroberfläche

- Als Spieler möchte ich das Spiel starten können, weil ich direkt mit dem Spielen beginnen möchte.
- Als Spieler möchte ich zwischen verschiedenen Leveln wählen können, weil ich gezielt bestimmte Spielabschnitte spielen möchte.
- Als Spieler möchte ich nach einem Game Over ins Hauptmenü zurückkehren können, weil ich eine neue Spielrunde starten oder andere Menüpunkte aufrufen möchte.
- Als Spieler möchte ich Highscores einsehen können, weil ich meine bisherigen Leistungen nachvollziehen möchte.
- Als Spieler möchte ich die Lautstärke anpassen können, weil ich das Spielerlebnis an meine persönlichen Vorlieben anpassen möchte.
- Als Spieler möchte ich die Steuerung konfigurieren können, weil unterschiedliche Spieler verschiedene Eingabegeräte und Vorlieben besitzen.

Gameplay

- Als Spieler möchte ich meine Spielfigur bewegen können, weil ich mich durch das Level navigieren möchte.
- Als Spieler möchte ich springen und einen Doppelsprung ausführen können, weil ich Hindernisse überwinden und schwer erreichbare Plattformen erreichen möchte.
- Als Spieler möchte ich Gegner bekämpfen können, weil sie meinen Fortschritt im Level behindern und Punkte liefern.
- Als Spieler möchte ich Leben verlieren können, weil das Spiel eine Herausforderung darstellen soll.
- Als Spieler möchte ich Checkpoints aktivieren können, weil ich nach einem Tod nicht das gesamte Level erneut spielen möchte.

- Als Spieler möchte ich das Level erfolgreich abschließen können, weil ich im Spiel fortschreiten möchte.

Fortschrittssystem

- Als Spieler möchte ich meinen aktuellen Punktestand sehen können, weil ich meine Leistung während des Spiels bewerten möchte.
- Als Spieler möchte ich gespeicherte Highscores einsehen können, weil ich meine langfristigen Fortschritte verfolgen möchte.
- Als Spieler möchte ich meine beste Levelzeit speichern können, weil ich meine Leistung mit früheren Versuchen vergleichen möchte.
- Als Spieler möchte ich meinen Fortschritt innerhalb eines Levels erkennen können, weil ich abschätzen möchte, wie weit ich bereits gekommen bin.

Power-Ups

- Als Spieler möchte ich Power-Ups einsammeln können, weil sie mir zeitweise Vorteile im Spiel verschaffen.
- Als Spieler möchte ich Schutzschilde verwenden können, weil ich dadurch Schaden vermeiden kann.
- Als Spieler möchte ich Magneten verwenden können, weil ich Münzen einfacher einsammeln kann.
- Als Spieler möchte ich Stern-Power-Ups verwenden können, weil ich dadurch vorübergehend unverwundbar werde.
- Als Spieler möchte ich Feuer-Power-Ups verwenden können, weil ich zusätzliche Angriffsmöglichkeiten erhalten möchte.

Entwicklerperspektive

- Als Entwickler möchte ich den Quellcode modular strukturieren, weil dadurch die Wartbarkeit verbessert wird.
- Als Entwickler möchte ich eine übersichtliche Projektstruktur verwenden, weil Erweiterungen und Anpassungen dadurch einfacher umgesetzt werden können.
- Als Entwickler möchte ich Debug-Funktionen einsetzen können, weil Fehler dadurch schneller identifiziert und behoben werden können.
- Als Entwickler möchte ich Spielstände und Einstellungen persistent speichern können, weil Benutzerdaten auch nach dem Neustart der Anwendung erhalten bleiben sollen.

3.5 Priorisierung der Anforderungen

Zur Priorisierung der Anforderungen wurde die MoSCoW-Methode verwendet.

Muss-Anforderungen

- Bewegung der Spielfigur
- Sprungsystem
- Kollisionserkennung
- Levelstruktur

- Gegnerinteraktionen
- Punktesystem

Soll-Anforderungen

- Highscore-System
- Checkpoints
- Benutzeroberflächen
- Soundeffekte
- Mehrere Level

Kann-Anforderungen

- Charakterauswahl
- Zusätzliche Power-Ups
- Erweiterte Audioeffekte
- Weitere Level und Spielinhalte

Durch die Priorisierung konnte sichergestellt werden, dass zu jedem Zeitpunkt eine lauffähige und spielbare Version des Systems verfügbar war. Erweiterte Funktionen wurden schrittweise ergänzt, ohne die Kernfunktionalität des Spiels zu beeinträchtigen.

4 Use-Case-Modellierung

Zur Beschreibung der funktionalen Anforderungen wurden Use Cases verwendet. Diese beschreiben typische Interaktionen zwischen dem Spieler und dem System. Auf Basis der Use Cases wurden anschließend Aktivitäts- und Sequenzdiagramme erstellt, um die Abläufe grafisch darzustellen.

4.1 Use Case: Spiel starten

Beschreibung

Dieser Anwendungsfall beschreibt den Ablauf vom Start des Spiels bis zum Laden eines Levels.

Akteur

Spieler

Vorbedingungen

- Das Spiel wurde gestartet.
- Das Hauptmenü ist geladen.

Nachbedingungen

- Das ausgewählte Level wurde geladen.
- Der Spieler befindet sich im aktiven Spiel.

Standardablauf

1. Der Spieler startet das Spiel.
2. Das System zeigt das Hauptmenü an.
3. Der Spieler betätigt den Button „Start“.
4. Das System spielt einen Bestätigungston ab.
5. Das System öffnet die Levelauswahl.
6. Der Spieler wählt ein Level aus.
7. Das System lädt die entsprechende Spielszene.
8. Das Spiel beginnt.

Alternative Abläufe**A1 – Einstellungen öffnen**

- Der Spieler wählt den Menüpunkt „Settings“.
- Das System öffnet das Einstellungsmenü.

A2 – Highscores anzeigen

- Der Spieler wählt den Menüpunkt „Highscores“.
- Das System öffnet die Highscore-Ansicht.

A3 – Spiel beenden

- Der Spieler wählt den Menüpunkt „Exit“.
- Das System zeigt einen Bestätigungsdialog an.
- Nach Bestätigung wird das Spiel beendet.

Aktivitätsdiagramm

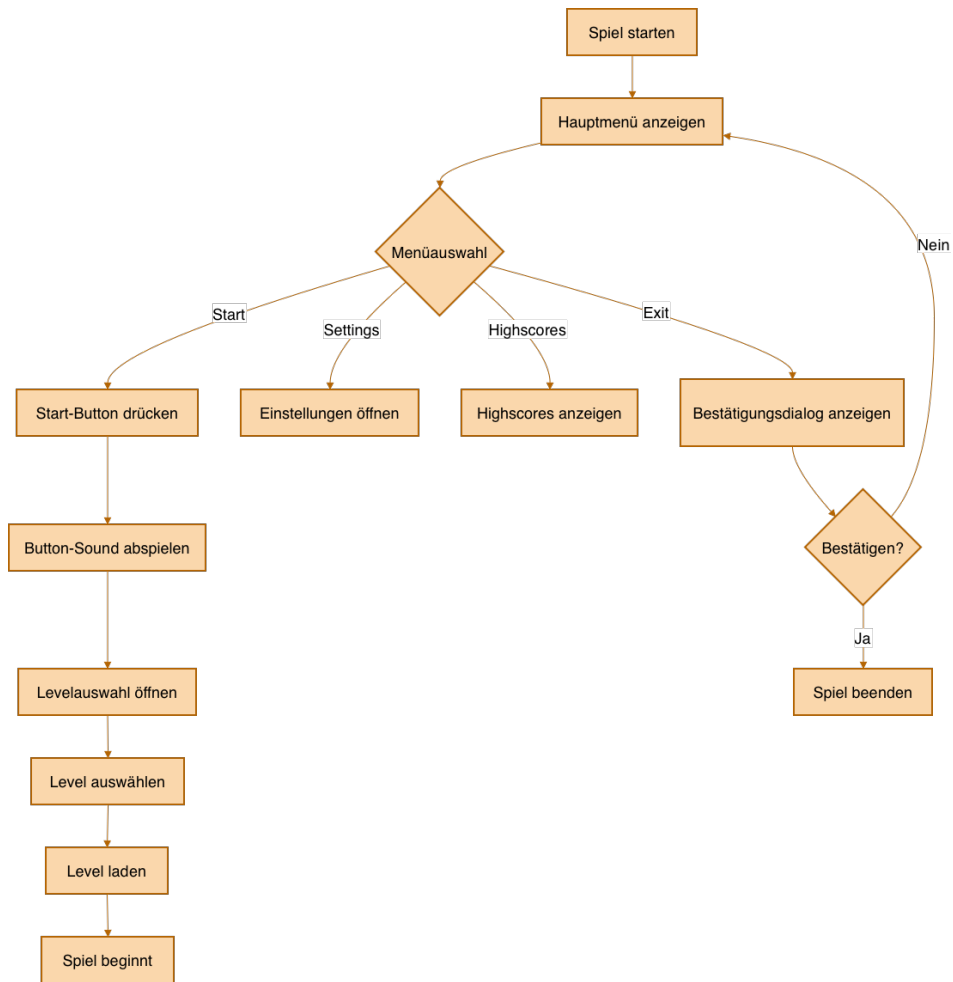


Abbildung 1: Aktivitätsdiagramm: Use Case – Spiel starten

Sequenzdiagramm

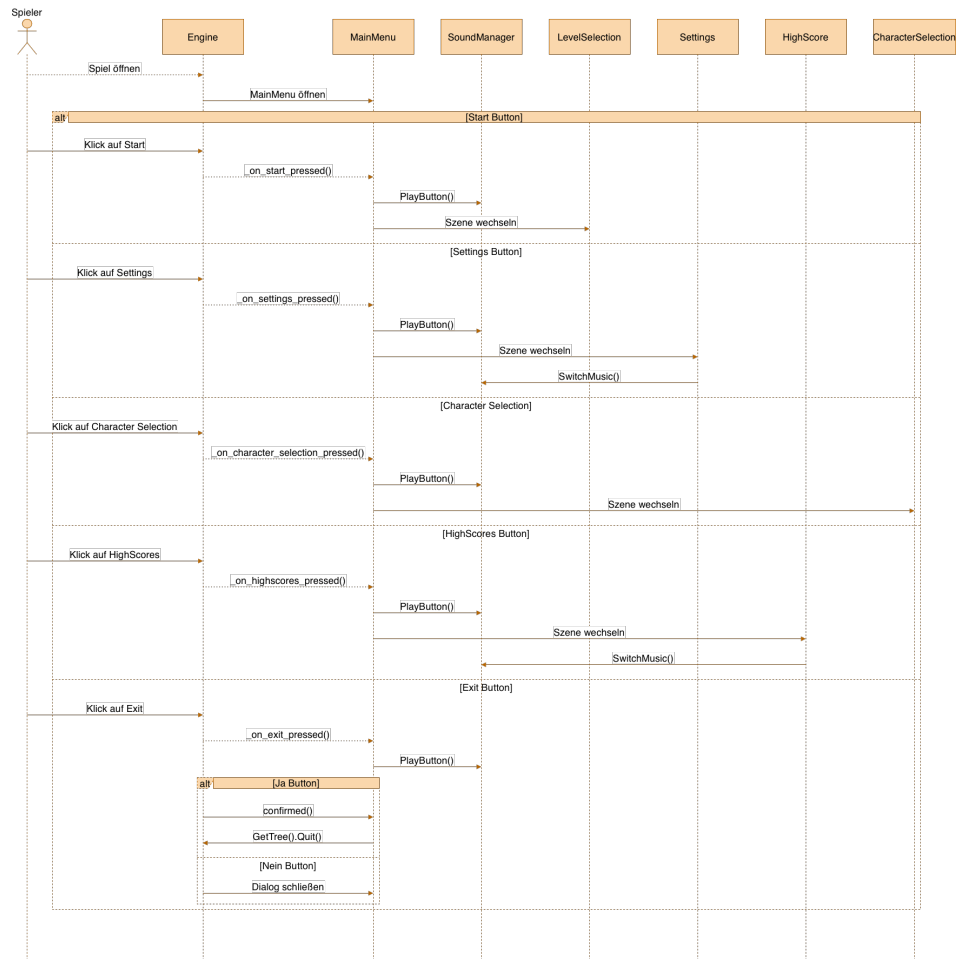


Abbildung 2: Sequenzdiagramm: Use Case – Spiel starten

4.2 Use Case: Gegnerkontakt

Beschreibung

Dieser Anwendungsfall beschreibt die Reaktion des Systems, wenn der Spieler mit einem Gegner kollidiert.

Akteur

Spieler

Vorbedingungen

- Ein Level ist aktiv.
- Der Spieler befindet sich im Spiel.
- Ein Gegner befindet sich innerhalb der Spielwelt.

Nachbedingungen

- Der Gegner wurde besiegt, oder
- der Spieler verliert Leben bzw. wird beschädigt, oder
- der Game-Over-Bildschirm wird angezeigt.

Standardablauf

1. Der Spieler bewegt sich durch das Level.
2. Der Spieler berührt einen Gegner.
3. Das System erkennt die Kollision.
4. Das System überprüft den aktuellen Zustand des Spielers.
5. Der Spieler erhält Schaden.
6. Das HUD wird aktualisiert.
7. Der Spieler wird gegebenenfalls zum letzten Checkpoint zurückgesetzt.
8. Das Spiel wird fortgesetzt.

Alternative Abläufe

A1 – Spieler besitzt Star-Power

- Der Spieler kollidiert mit einem Gegner.
- Das System erkennt die aktive Star-Power.
- Der Gegner wird sofort besiegt.
- Der Spieler erhält keinen Schaden.

A2 – Spieler besitzt ein Schild

- Der Spieler kollidiert mit einem Gegner.
- Das System erkennt ein aktives Schild.
- Das Schild wird entfernt.
- Der Spieler bleibt unverletzt.

A3 – Gegner wird von oben besprungen

- Der Spieler springt auf einen Gegner.
- Das System erkennt einen Stomp-Angriff.
- Der Gegner wird besiegt.
- Der Spieler erhält Punkte.

A4 – Keine Leben mehr vorhanden

- Der Spieler verliert sein letztes Leben.
- Das System speichert den Punktestand.
- Der Game-Over-Bildschirm wird geladen.

Aktivitätsdiagramm

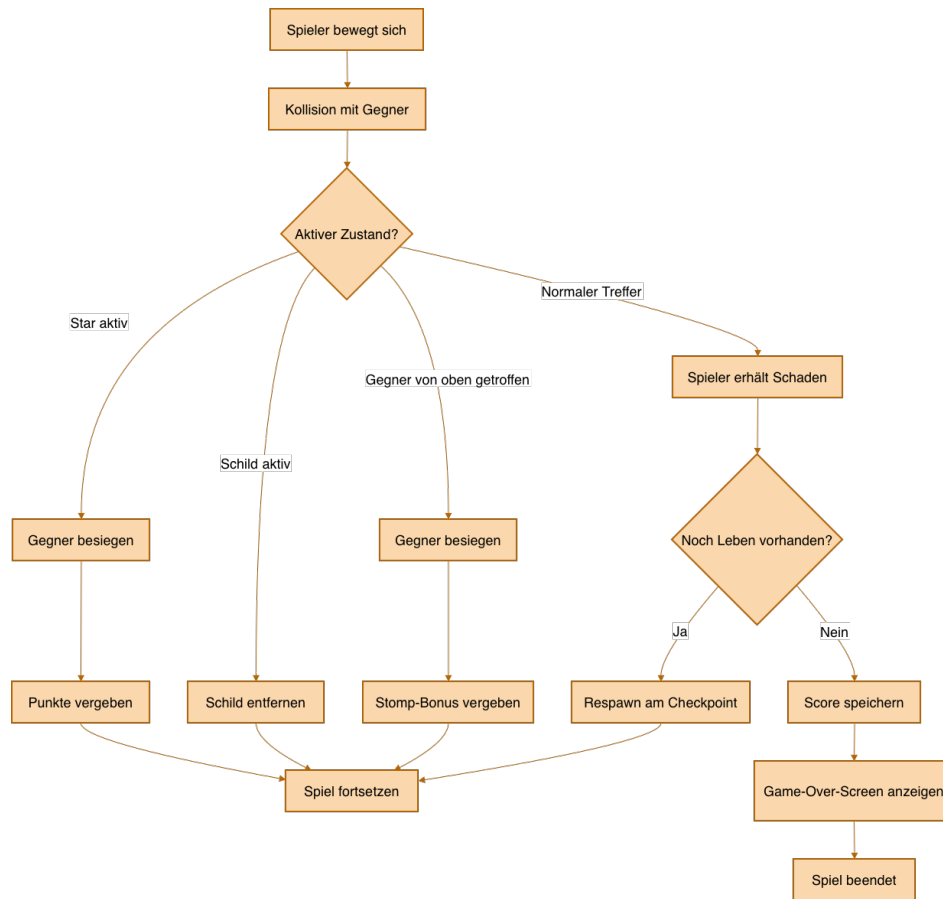


Abbildung 3: Aktivitätsdiagramm: Use Case – Gegnerkontakt

Sequenzdiagramm

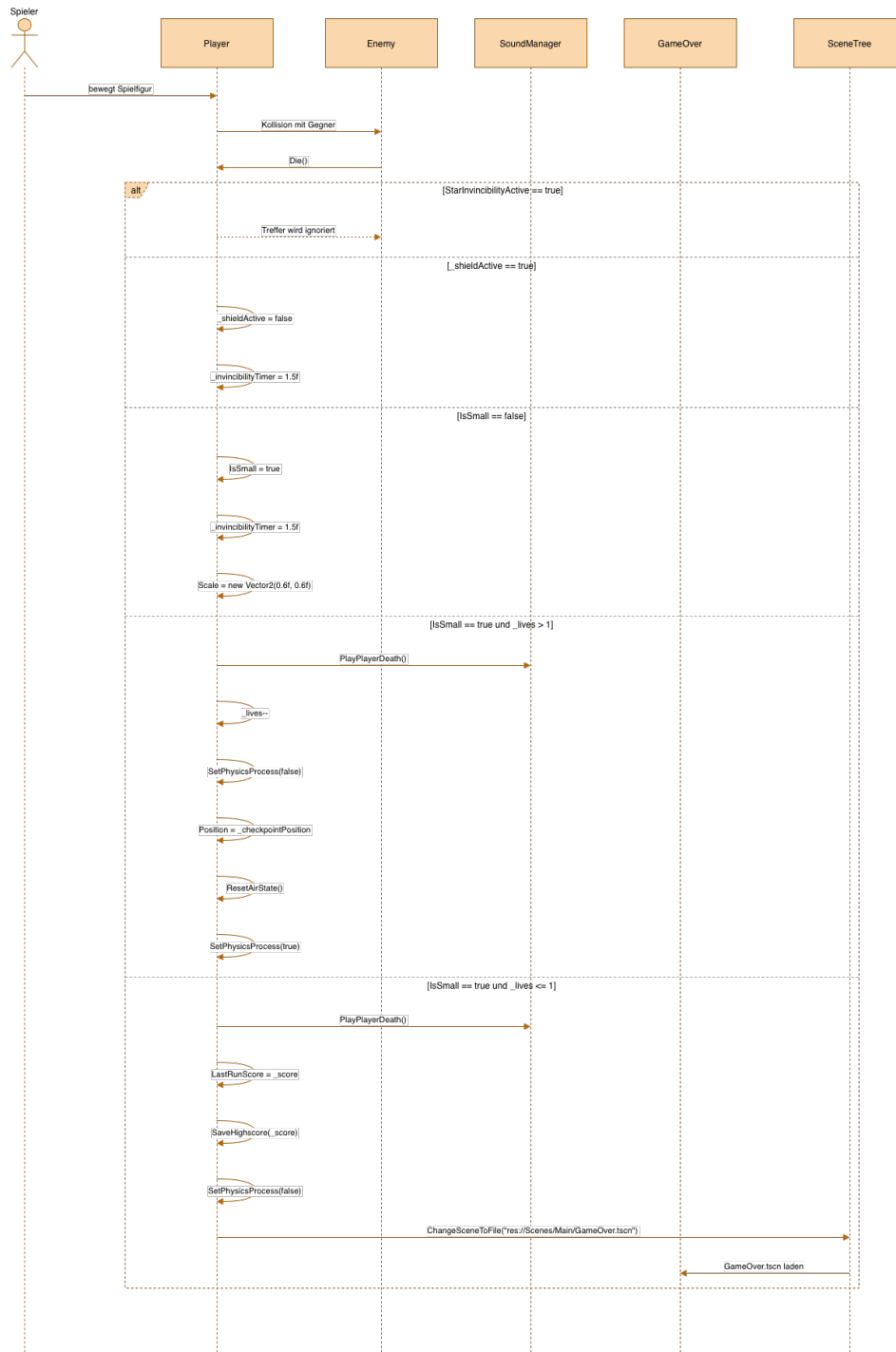


Abbildung 4: Sequenzdiagramm: Use Case – Gegnerkontakt

5 Systementwurf (Design)

5.1 Architekturüberblick

Die Architektur orientiert sich an einer klaren Trennung zwischen Benutzeroberfläche, Spiellogik und unterstützenden Systemdiensten. Ziel ist eine bessere Wartbarkeit, Erweiterbarkeit und Nachvollziehbarkeit des Systems.

Die Benutzeroberfläche umfasst sämtliche Menüs und Anzeigen. Hierzu zählen insbesondere das Hauptmenü, das Pausenmenü, die Einstellungen, die Highscore-Anzeige, die Charakterauswahl sowie das Heads-Up-Display (HUD). Diese Komponenten dienen der Interaktion mit dem Spieler und der Darstellung relevanter Informationen.

Die Spiellogik bildet den Kern des Systems. Zu den wichtigsten Komponenten gehören die Klassen **Player**, **Enemy**, **Projectile** und **LevelGoal**. Diese Klassen verwalten Bewegungen, Kollisionen, Spielzustände, Punktesysteme, Power-Ups sowie den Fortschritt innerhalb eines Levels.

Zusätzliche Systemdienste werden durch den **SoundManager** sowie die Speicherung von Profil-, Einstellungs- und Highscore-Daten bereitgestellt. Diese Komponenten stellen Funktionen bereit, die von mehreren Bereichen des Systems gemeinsam genutzt werden.

Die Godot Engine übernimmt dabei die Verwaltung von Szenen, die Physiksimulation, die Kollisionsberechnung, die Eingabeverarbeitung sowie den Lebenszyklus der Spielobjekte.

Architekturansatz

Während der Entwurfsphase wurde die Verwendung des Model-View-Controller-(MVC)-Musters betrachtet. Aufgrund der Godot Engine wurde jedoch auf eine strikte Umsetzung verzichtet.

Godot basiert auf einer szenen- und komponentenorientierten Architektur, bei der Spielobjekte als Nodes innerhalb einer Szenenhierarchie organisiert werden. Die Engine stellt bereits zentrale Mechanismen für Ereignisverarbeitung, Szenenverwaltung, Physik und Eingaben bereit.

Für das vorliegende Projekt erwies sich daher eine modulare Aufteilung in eigenständige Komponenten als sinnvoller. Die einzelnen Spielobjekte kapseln ihre jeweilige Funktionalität weitgehend selbstständig.

Architekturdiagramm

Aufgrund der Komplexität und Größe des Architekturdiagramms ist dieses als separate Datei beigelegt (Architekturdiagramm.drawio.png). Eine verkleinerte Darstellung im Dokument würde die Lesbarkeit erheblich einschränken. Wir empfehlen daher die Betrachtung der separaten Datei für eine vollständige Übersicht aller Schichten und Komponenten.

5.2 Komponenten- und Moduldesign

Das System wurde in mehrere logisch getrennte Module untergliedert. Die folgende Tabelle gibt einen Überblick über die wichtigsten Klassen und ihre Verantwortlichkeiten.

Klasse / Modul	Verantwortlichkeit
Player	Bewegung, Sprung, Kampf und persistente Profildaten der Spielfigur
Enemy	Verhalten aller fünf Gegnertypen, Kollision und Schadenslogik
Projectile	Geschosse von Gegnern und Spieler, inklusive Zurückschlagen
LevelGoal	Erkennung des Levelabschlusses und Zeitbonus
GameManager	Pause-Steuerung, Levelstart und Musikwechsel
SoundManager	Zentrale Verwaltung von Musik und Soundeffekten (Singleton)
HUD	Anzeige von Leben, Punkten, Zeit und Power-Up-Status
Checkpoint	Speicherung der Wiedereinstiegsposition
AudioUtils	Umrechnung zwischen Lautstärke-Prozent und Dezibel

Tabelle 2: Übersicht der Klassen und Verantwortlichkeiten

Player

Die Klasse **Player** stellt die zentrale Spielfigur dar und ist als partielle Klasse über drei Dateien verteilt:

```

1 public partial class Player : CharacterBody2D
2 {
3     // Player.cs           -> Bewegung, Sprung, Zustand
4     // Player.Combat.cs    -> Schwert, Feuerball, Angriff
5     // Player.Profile.cs   -> Coins, Charakterwahl, Highscore
6 }

```

Listing 1: Aufteilung der Spielfigur in eine partielle Klasse

Enemy und Projectile

Die Klasse **Enemy** repräsentiert die Gegner und verwaltet Bewegungsmuster, Kollisionen sowie die Reaktion auf Angriffe. Über den Aufzählungstyp **EnemyType** werden fünf Verhaltensweisen abgebildet (**Patrol**, **Fast**, **Jumping**, **Charger**, **Shooter**). Die Klasse **Projectile** verwaltet Geschosse – sowohl gegnerische Projektile als auch vom Spieler erzeugte Feuerbälle.

Power-Ups und Pickups

Die sammelbaren Objekte wurden als eigenständige Komponenten umgesetzt: **ShieldPowerUp**, **StarPowerUp**, **MagnetPowerUp**, **GrowthPickup**, **SwordPickup** und **HeartPickup**.

HUD und SoundManager

Das HUD stellt Punktestand, verbleibende Leben, Spielzeit, aktive Power-Ups und den Levelfortschritt dar. Der **SoundManager** ist als Autoload eingerichtet und übernimmt die zentrale Verwaltung aller Musik- und Soundeffekte, sodass die Audiosteuerung unabhängig von einzelnen Szenen erfolgt.

Menüsystem

Das Menüsystem umfasst **MainMenu**, **PauseMenu**, **Settings**, **Volume**, **HighScores** und **CharacterSelection**.

5.3 UML-Diagramme

5.3.1 Klassendiagramm

Das Klassendiagramm beschreibt die wichtigsten Klassen des Systems sowie deren Beziehungen untereinander. Im Zentrum steht die Klasse **Player**. **Player**, **Enemy** und die beweglichen Plattformen

erben von `CharacterBody2D` und nutzen damit das Physik- und Kollisionssystem der Engine. Sämtliche Sammelobjekte und Power-Ups erben von `Area2D`. Die Klasse `Enemy` hält den Typ `EnemyType` und erzeugt bei Bedarf Instanzen von `Projectile`. Der `SoundManager` wird von nahezu allen Klassen über seine statische Instanz angesprochen.

Aufgrund der Komplexität und Größe des Klassendiagramms ist dieses als separate Datei beigefügt (Klassendiagramm.drawio.png). Eine verkleinerte Darstellung im Dokument würde die Lesbarkeit erheblich einschränken. Wir empfehlen daher die Betrachtung der separaten Datei für eine vollständige Übersicht aller Klassen und Beziehungen.

5.3.2 Zustandsdiagramm

Für die Spielfigur wurde ein Zustandsdiagramm erstellt, das die wichtigsten Zustände des Players sowie die Übergänge zwischen diesen darstellt.

Beispielhafte Zustände: Normal, Small, Shield, Star, Dying, Resawning, GameOver.

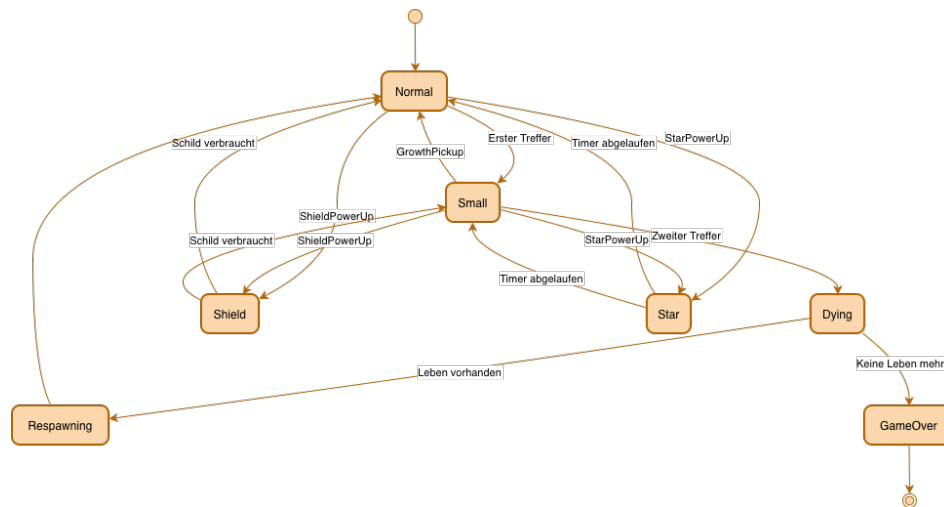


Abbildung 5: Zustandsdiagramm: Player

5.4 Zentrale Spielobjekte

5.4.1 Gegner

In diesem Spiel gibt es fünf verschiedene Arten von Gegnern, die mithilfe eines Enums (`EnemyType`) definiert wurden. Alle Gegnertypen können durch einen Sprung von oben (Stomp), durch die Sternen-Unverwundbarkeit oder durch einen Schwertangriff besiegt werden.

Patrol: Bewegt sich mit normaler Geschwindigkeit kontinuierlich zwischen zwei Punkten. Dient als grundlegender Standardgegner.

Fast: Verhält sich wie ein Patrol-Gegner, bewegt sich jedoch mit einer deutlich höheren Geschwindigkeit von 220 Einheiten pro Sekunde.

Jumping: Patrouilliert zwischen zwei Punkten und führt zusätzlich in regelmäßigen Abständen Sprünge aus. Dadurch entsteht ein weniger vorhersehbares Bewegungsmuster.

Charger: Verfolgt den Spieler aktiv, sobald dieser sich innerhalb eines Radius von etwa 350 Pixeln befindet. Bewegt sich mit 380 Einheiten pro Sekunde direkt auf den Spieler zu. Außerhalb dieses Bereichs stoppt er allmählich.

Shooter: Bleibt stationär und greift den Spieler aus der Distanz an. Sobald der Spieler sich innerhalb seiner Sichtweite befindet, feuert er in regelmäßigen Intervallen Projektile in dessen Richtung. Im Gegensatz zum Charger bewegt er sich nicht aktiv auf den Spieler zu, sondern stellt eine dauerhafte Bedrohung aus sicherer Entfernung dar. Gegnerische Projektile können durch einen präzise getimten Schwertschlag zurückgeschlagen werden und treffen dann den ursprünglichen Schützen.

5.4.2 Level Objects

Innerhalb der Levels gibt es verschiedene Objekte, die als eigene Szenen mit separaten Skripten implementiert wurden.

Moving Platform: Bewegliche Plattform, die sich kontinuierlich zwischen zwei Positionen hin- und herbewegt. Die Bewegung erfolgt mithilfe einer Sinusfunktion, wodurch die Plattform an den Wendepunkten sanft abbremst.

Crushing Platform: Spezielle Variante der beweglichen Plattform mit gefährlichen Stacheln. Berührt der Spieler diese Stacheln von oben, wird er sofort besiegt.

Spike: Stationäre Gefahrenobjekte. Berührt der Spieler die Stacheln, verliert er unmittelbar ein Leben.

Checkpoint: Sobald der Spieler einen Checkpoint berührt, wird dessen Position als neuer Respawn-Punkt gespeichert. Der aktivierte Checkpoint verändert seine Farbe zu Grün.

5.4.3 Pickups

Pickups sind Levelobjekte, die vom Spieler eingesammelt werden können.

Coin: Erhöht den Punktestand um eins. Coins werden dauerhaft gespeichert und können im Shop für neue Charaktere verwendet werden.

Growth Pickup: Ermöglicht dem Spieler, nach einem Treffer von der kleinen in die große Form zurückzukehren.

Star-Power-Up: Aktiviert den Sternenmodus für 6 Sekunden und eine Unverwundbarkeit von 6,5 Sekunden. Wechselt die Hintergrundmusik auf die Sternenmusik.

Sword Pickup: Füllt die verfügbaren Schwertladungen auf. Mit dem Schwert können Gegner besiegt sowie feindliche Projektile abgewehrt werden.

Fire Flower: Verleiht dem Spieler die Fähigkeit, Feuerbälle abzufeuern. Verbraucht ebenfalls eine Schwertladung. Nach dem Einsammeln wird der Feuer-Modus aktiviert.

6 Implementierung

6.1 Verwendete Technologien und Tools

Für die Entwicklung des Spiels wurde die Game Engine Godot in Kombination mit der Programmiersprache C# verwendet. Godot wurde gewählt, da die Engine eine einfache Szenenstruktur, ein integriertes Physiksystem sowie eine gute Unterstützung für 2D-Spiele bietet.

Die Spiellogik wurde vollständig in C# implementiert. Dabei kamen objektorientierte Konzepte wie Vererbung, Kapselung und Klassenaufteilung zum Einsatz. Zur Versionsverwaltung wurde Git verwendet. Zur Qualitätssicherung wurde zusätzlich das Unit-Test-Framework NUnit eingesetzt.

Zusätzlich wurden verschiedene Werkzeuge und Systeme der Godot Engine genutzt:

- Szenensystem zur Strukturierung des Spiels
- Physik- und Kollisionssystem

- Audio- und Musiksysteem
- UI-System für Menüs und HUD
- Input-System für Tastatursteuerung
- Dateisystem zur Speicherung von Highscores und Einstellungen

6.2 Aufbau der Codebasis

Das Projekt folgt einer nach Zuständigkeit gegliederten Ordnerstruktur. Quellcode, Szenen und Medieninhalte sind getrennt abgelegt.

```
Project
|-- Scenes
|   |-- Levels
|   |-- Main
|   |-- Pickups
|   |-- Enemies
|   '-- level_objects
|-- Scripts
|   |-- Player
|   |-- UI
|   |-- Enemies
|   |-- Pickups
|   '-- Managers
|-- Assets
|   |-- Sounds
|   |-- Music
|   '-- Sprites
'-- Tests
```

Listing 2: Projektstruktur

Persistente Daten werden in lokalen Dateien im Nutzerverzeichnis abgelegt:

Datei	Klasse	Inhalt
profile.cfg	ConfigFile	Münzstand, gewählte und freigeschaltete Charaktere
settings.cfg	ConfigFile	Tastenbelegung, Lautstärke je Audio-Bus
highscore.dat	FileAccess	Höchster erreichter Punktestand
level1_time.dat	FileAccess	Beste Zeit für das Level

Tabelle 3: Persistente Datenspeicherung

```
1 // Strukturierte Daten (profile.cfg)
2 var config = new ConfigFile();
3 config.Load("user://profile.cfg");
4 Coins = (int)config.GetValue("currency", "coins", 0);
5
6 // Einfacher Binaerwert (highscore.dat)
7 using var file = FileAccess.Open("user://highscore.dat",
8                                 FileAccess.ModeFlags.Write);
9 file.Store32((uint)score);
```

Listing 3: Persistenz über ConfigFile und FileAccess

6.3 Zentrale Implementierungsaspekte

6.3.1 Spielersteuerung und Bewegung

Die Spielersteuerung wurde mit besonderem Augenmerk auf ein präzises und reaktionsschnelles Spielgefühl entwickelt. Die Figur basiert auf einem `CharacterBody2D` und wird über `MoveAndSlide()` in der Physik-Schleife bewegt. Horizontale Bewegung erfolgt über Beschleunigung und Abbremsung, wobei zwischen Boden- und Luftbeschleunigung sowie einem gesonderten Wert für Richtungswechsel unterschieden wird.

Parameter	Wert	Bedeutung
MaxSpeed	440	Maximale horizontale Geschwindigkeit
Acceleration	650	Beschleunigung am Boden
AirAcceleration	1000	Beschleunigung in der Luft
TurnAcceleration	2500	Beschleunigung bei Richtungswechsel
JumpVelocity	-600	Anfangsgeschwindigkeit des Sprungs
JumpGravity	700	Schwerkraft während des Aufstiegs
FallGravity	1800	Schwerkraft beim Fallen
CoyoteTime	0,10 s	Sprungfenster nach Verlassen einer Kante
JumpBufferTime	0,12 s	Vorgemerakter Sprung vor der Landung

Tabelle 4: Bewegungsparameter der Spielfigur

Das Sprungsystem verwendet eine asymmetrische Gravitation:

```

1 float gravity = (velocity.Y < 0 && Input.IsActionPressed("jump")
2   && JumpHoldTimer > 0f)
3   ? JumpGravity      // Aufstieg, Taste gehalten -> hoeher springen
4   : FallGravity;     // Fall oder Taste losgelassen -> schneller fallen
5
6 if (!IsOnFloor()) velocity.Y += gravity * dt;
```

Listing 4: Asymmetrische Gravitation für variable Sprunghöhe

Ergänzt wird dies durch *Coyote Time* und *Jump Buffer*. Die Standardbelegung lautet:

Aktion	Standardtaste
Nach links bewegen	A
Nach rechts bewegen	D
Springen	Leertaste
Ducken	S
Angreifen	J

Tabelle 5: Standardtastenbelegung

6.3.2 Gegner

Gegnertyp	Verhalten
Patrol	Läuft zwischen zwei Punkten hin und her (Standardgegner)
Fast	Wie Patrol, jedoch mit deutlich erhöhter Geschwindigkeit
Jumping	Springt in regelmäßigen Abständen
Charger	Sprintet auf den Spieler zu, sobald dieser in Reichweite kommt
Shooter	Bleibt stehen und feuert in Intervallen Projektile in Richtung des Spielers

Tabelle 6: Gegnertypen und ihr Verhalten

Ein Stomp-Chain-System belohnt das Besiegen mehrerer Gegner in Folge während eines Sprungs mit zusätzlichen Punkten.

6.3.3 Kampfsystem

Das Kampfsystem basiert auf einem Nahkampfangriff mit einem Schwert. Die Anzahl der Schwerthiebe ist begrenzt und kann über Pickups aufgefüllt werden. Mit aktivem Feuer-Power-Up erzeugt jeder Schwertschlag zusätzlich ein Projektil.

6.3.4 Power-Ups und Sammelobjekte

Objekt	Wirkung
Shield	Absorbiert einen einzelnen Treffer und bricht anschließend
Star	Macht den Spieler für kurze Zeit unverwundbar
Magnet	Zieht in der Nähe befindliche Münzen zur Figur
Fire Flower	Verleiht Fernkampfangriffe in Form von Feuerbällen
Growth-Pickup	Lässt die geschrumpfte Figur auf normale Größe wachsen
Heart-Pickup	Gewährt ein zusätzliches Leben
Sword-Pickup	Füllt die verfügbaren Schwerthiebe wieder auf

Tabelle 7: Power-Ups und Sammelobjekte

6.3.5 Fortschritt, Leben und Persistenz

Der Punktestand wird durch das Sammeln von Münzen und das Besiegen von Gegnern erhöht. Bei Erreichen bestimmter Punkteschwellen wird ein zusätzliches Leben gewährt. Checkpoints speichern die aktuelle Position für den Respawn-Punkt.

6.3.6 Audio- und Effektsystem

Die gesamte Audiosteuerung läuft über den als Singleton umgesetzten **SoundManager**. Er ermöglicht das nahtlose Wechseln der Hintergrundmusik je nach Spielzustand. Die Umrechnung zwischen prozentualer Lautstärke und Dezibelwerten wurde in die Hilfsklasse **AudioUtils** ausgelagert, um sie testbar zu halten.

6.3.7 Benutzeroberfläche

Das HUD zeigt die verbleibenden Leben, den Punktestand, die verstrichene Zeit, die verfügbaren Schwerthiebe sowie die verbleibende Dauer aktiver Power-Ups an.

6.3.8 Verwendete Entwurfsmuster

Singleton Der `SoundManager` wurde als Singleton umgesetzt (`PlayButton()`, `PlayCoin()`, `PlayJump()`, `SwitchMusic()`).

Komponentenorientierter Aufbau Spielobjekte wie Gegner, Power-Ups und Plattformen wurden als eigenständige Szenen mit zugehörigen Skripten umgesetzt.

Factory-ähnliche Objekterzeugung Objekte werden dynamisch zur Laufzeit erzeugt, z. B. spawnen `ItemBlocks` unterschiedliche Objekte oder Gegner erzeugen Projektile.

Ereignisgesteuerte Kommunikation Kommunikation erfolgt über Godot-Signale: `BodyEntered`, `MouseEntered`, `Pressed`, `Confirmed`.

6.4 Herausforderungen während der Umsetzung

Keines der Teammitglieder hatte zuvor Erfahrungen im Bereich Videospielentwicklung gemacht. Besonders im Bereich der Versionsverwaltung mit Git kam es zeitweise zu Merge-Konflikten. Das Balancing der Spielmechaniken erforderte viele Testläufe, insbesondere für Bewegungsparameter, Sprungverhalten, Gegnergeschwindigkeiten und Power-Up-Dauer.

7 Qualitätssicherung und Test

7.1 Teststrategie

Die Qualitätssicherung erfolgte durch eine Kombination aus isolierten Unit-Tests einzelner Komponenten sowie manuellen Systemtests des gesamten Spiels. Die Tests wurden regelmäßig am Ende jedes Sprints durchgeführt.

7.2 Unit-Testplan mit Äquivalenzklassen

Für die Implementierung der Tests wurde das Framework NUnit verwendet. Im Rahmen der Testimplementierung wurden die Methoden `PercentToDb()` und `DbToPercent()` der Klasse `AudioUtils` betrachtet.

Äquivalenzklassen für `PercentToDb()`

Äquivalenzklasse	Wert	Erwartetes Ergebnis
Gültiger Minimalwert	0 %	−80 dB (Stummschaltung)
Gültiger Maximalwert	100 %	0 dB
Gültiger Zwischenwert	50 %, 75 %	Negativer dB-Wert im gültigen Bereich
Ungültig unterhalb	−50 %	Clamping auf Minimalwert
Ungültig oberhalb	200 %	Clamping auf Maximalwert

Tabelle 8: Äquivalenzklassen für `PercentToDb()`

Äquivalenzklassen für DbToPercent()

Äquivalenzklasse	Wert	Erwartetes Ergebnis
Minimalwert	−80 dB	0 %
Maximalwert	0 dB	100 %
Zwischenwert	−10 dB	Wert unter 100 %

Tabelle 9: Äquivalenzklassen für DbToPercent()

7.3 Testfallermittlung mittels Äquivalenzklassen

Die Auswahl der Testfälle erfolgte auf Grundlage der Äquivalenzklassenbildung. Zusätzlich wurden Roundtrip-Tests implementiert: Ein Ausgangswert wird mit `PercentToDb()` in Dezibel umgerechnet und anschließend über `DbToPercent()` zurückkonvertiert, um Rundungsfehler zu erkennen.

7.4 Erweiterbarkeit des Testkonzepts

Die implementierten Unit-Tests stellen einen exemplarischen Ausschnitt dar und können im Verlauf der Weiterentwicklung um Grenzwertanalysen, weitere Kernkomponenten oder automatische Regressionstests ergänzt werden.

7.5 Testfälle und Durchführung

ID	Testfall	Erwartet	Tatsächlich	Status	Anmerkung
T01	Spiel starten	Hauptmenü wird angezeigt	Hauptmenü erscheint fehlerfrei	Bestanden	–
T02	Neues Spiel starten	Level wird geladen	Level startet ohne Fehler	Bestanden	–
T03	Spieler nach rechts	Spieler läuft rechts	Bewegung flüssig	Bestanden	–
T04	Spieler nach links	Spieler läuft links	Bewegung flüssig	Bestanden	–
T05	Springen	Spieler springt korrekt	Sprunganimation und Physik korrekt	Bestanden, nachgebessert	Sprunghöhe initial zu hoch; JumpVelocity von -1000 auf -600 angepasst
T06	Ducken	Spieler geht in Duckposition	Duckanimation korrekt	Bestanden, nachgebessert	Spieler wuchs in Decken hinein – Kollisionscheck ergänzt
T07	Coin einsammeln	Coin verschwindet, Punkte steigen	Coin entfernt, Score erhöht	Bestanden	–
T08	Gegner berühren	Spieler verliert Leben	Leben reduziert	Bestanden, nachgebessert	Knockback fehlte initial; in Sprint 3 ergänzt
T09	Alle Leben verloren	Game-Over erscheint	Game-Over korrekt	Bestanden	–
T10	Level abschließen	LevelCompleteScreen	Bildschirm eingeblendet	Bestanden	–
T11	Punkteanzeige	HUD aktualisiert Punkte	Punkte sofort angezeigt	Bestanden	–
T12	Lebensanzeige	HUD aktualisiert Leben	Anzeige stimmt überein	Bestanden	–
T13	Timer	Zeit läuft kontinuierlich	Timer fehlerfrei	Bestanden	–
T14	Pause-Menü öffnen	Spiel pausiert	Spiel stoppt, Menü erscheint	Bestanden	–
T15	Pause-Menü schließen	Spiel läuft weiter	Spiel fortgesetzt	Bestanden	–
T16	Einstellungen öffnen	Einstellungsmenü erscheint	Menü geladen	Bestanden	–
T17	Musiklautstärke	Musik wird angepasst	Lautstärke ändert sich sofort	Bestanden	–

ID	Testfall	Erwartet	Tatsächlich	Status	Anmerkung
T18	Soundeffekte	Effekte angepasst	Änderungen übernommen	Bestanden	–
T19	Helligkeit über Slider	Bildschirmhelligkeit angepasst	Helligkeit ändert sich flüssig	Bestanden	–
T20	Button-Hover	Hover-Sound abgespielt	Hover-Sound korrekt	Bestanden	–
T21	Button anklicken	Klick-Sound abgespielt	Sound korrekt	Bestanden	–
T22	Charakter auswählen	Charakter übernommen	Auswahl gespeichert	Bestanden, nachgebessert	Auswahl ging nach Neustart verloren; Speichern in profile.cfg ergänzt
T23	Gesperrrter Charakter	LockIcon angezeigt	Charakter nicht wählbar	Bestanden	–
T24	Szenenwechsel	Neue Szene geladen	Kein Absturz	Bestanden	–
T25	Neustart des Levels	Level zurückgesetzt	Alle Werte initialisiert	Bestanden	–
T26	Rückkehr Hauptmenü	Hauptmenü geladen	Rückkehr fehlerfrei	Bestanden	–
T27	Einstellungen speichern	Werte bleiben erhalten	Nach Neustart vorhanden	Bestanden	–
T28	Hintergrundmusik je Szene	Musik automatisch abgespielt	Korrekte Musik, keine Fehler	Bestanden	–
T29	Gegner besiegen	Gegner wird entfernt	Gegner verschwindet korrekt	Bestanden	–
T30	Angriff	Angriffsanimation	Animation und Treffer korrekt	Bestanden	–
T31	Power-up einsammeln	Power-up aktiviert	Spieler erhält Fähigkeit	Bestanden	–
T32	Respawn nach Tod	Spieler am Startpunkt	Respawn korrekt	Bestanden, nachgebessert	Spieler respawnnte am Levelanfang statt Checkpoint; in Sprint 4 behoben
T33	Animationen	Animationen wechseln korrekt	Alle Animationen passend	Bestanden	–
T34	Kollision Hindernisse	Kollision korrekt	Keine Durchdringungen	Bestanden, nachgebessert	Spieler steckte in schrägen Flächen; Kollisionsmaske angepasst
T35	Respawn Lebensverlust	Spieler an letzter Flagge	Korrekt am Checkpoint	Bestanden	–
T36	Anzeigemodus	Spiel passt Darstellung an	UI korrekt dargestellt	Bestanden	–
T37	Power-up endet	Spieler in Normalzustand	Power-up korrekt beendet	Bestanden	–
T38	Flagge aktivieren	Checkpoint gespeichert	Als Respawnpunkt gesetzt	Bestanden	–
T39	Fortschrittsanzeige	HUD aktualisiert	Levelfortschritt korrekt	Bestanden	–
T40	Herzanzeige	Herzsymbole korrekt	Nach Lebensverlust aktualisiert	Bestanden	–
T41	Angriffsladebalken	Balken zeigt Schläge	Anzeige korrekt	Bestanden	–
T42	Spielstand Levelabschluss	Zeit, Punkte, Leben übernommen	Korrekt angezeigt	Bestanden	–
T43	AudioBus-Einstellungen	Richtiger AudioBus	Lautstärke korrekt	Bestanden	–
T44	Spiel beenden	Anwendung beendet	Spiel schließt fehlerfrei	Bestanden	–
T45	Tastenbelegung	Frei konfigurierbar	Korrekt gespeichert	Bestanden	–
T46	Steuerung zurücksetzen	Standardtasten wiederhergestellt	Alle Belegungen korrekt	Bestanden	–
T47	Highscore öffnen	Highscore-Menü geladen	Liste korrekt angezeigt	Bestanden	–
T48	Level auswählen	Gewähltes Level startet	Level startet korrekt	Bestanden	–

ID	Testfall	Erwartet	Tatsächlich	Status	Anmerkung
T49	Charakter wechseln	Neuer Charakter übernommen	Auswahl korrekt	Bestanden	–
T50	Highscore speichern	Punktzahl dauerhaft gespeichert	Highscore nach Neustart angezeigt	Bestanden	–

Tabelle 10: Testfälle und Durchführung (T01–T50)

7.6 Testergebnisse und Analyse

Von insgesamt 50 durchgeführten Testfällen wurden alle erfolgreich bestanden. Bei 6 Testfällen (T05, T06, T08, T22, T32, T34) wurden während der Entwicklung Fehler identifiziert und behoben. Die Korrekturen betrafen hauptsächlich Physik- und Kollisionsverhalten sowie die Persistenz von Spielerdaten.

Die automatisierten NUnit-Tests für die Klasse `AudioUtils` liefen in allen CI-Durchläufen fehlerfrei durch. Die vollständigen Testergebnisse der GitHub Actions Pipeline sind direkt im Repository einsehbar:

<https://github.com/kurzmicahel02-hue/RunnerGame/actions>

7.7 Automatisierte Testausführung mittels GitHub Actions

Zur zusätzlichen Absicherung der Softwarequalität wurde eine CI-Pipeline mit GitHub Actions eingerichtet. Diese führt die Unit-Tests automatisch bei jedem Push sowie bei Pull-Requests aus.

Die Pipeline basiert auf einer Workflow-Datei (`.github/workflows/tests.yml`) und wird auf einem standardisierten Ubuntu-Runner ausgeführt. Dabei werden das Repository ausgecheckt, die .NET-Umgebung eingerichtet, die Abhängigkeiten wiederhergestellt, das Projekt kompiliert und alle NUnit-Tests ausgeführt.

Die Testergebnisse sind in GitHub direkt einsehbar und werden visuell als erfolgreich (✓) oder fehlgeschlagen (✗) dargestellt.

8 Projektmanagement

8.1 Vorgehensmodell

Für die Durchführung des Projekts wurde ein agiles Vorgehensmodell gewählt, das sich an Scrum orientiert. Zu Beginn wurden rund 100 User Stories erstellt und priorisiert. Obwohl sich der Entwicklungsstart dadurch zeitlich nach hinten verschob, erwies sich dieser Ansatz als vorteilhaft – viele Entscheidungen konnten bereits früh getroffen werden.

Die Entwicklung erfolgte iterativ in mehreren Sprints. Zur Aufwandsschätzung wurden Story Points mit den Werten 0, 1, 2, 3, 5, 8, 10 verwendet. Jedes Teammitglied übernahm dabei pro Sprint Aufgaben im Umfang von etwa 15 bis 20 Story Points.

Zur Verwaltung der Aufgaben wurde ein Kanban-Board mit den Kategorien „To Do“, „In Progress“ und „Done“ eingesetzt.



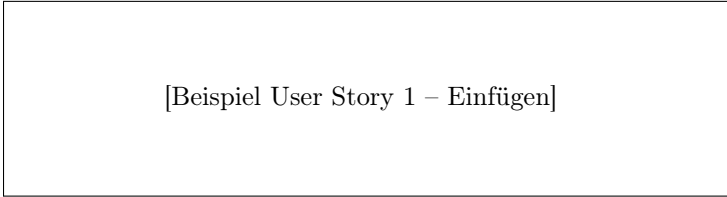
Abbildung 6: Kanban-Board während der aktiven Entwicklungsphase

Das Kanban-Board zeigt die Aufgabenverteilung während der aktiven Entwicklungsphase. Aufgaben wurden den Kategorien To Do, In Progress und Done zugeordnet und wöchentlich im Team aktualisiert.



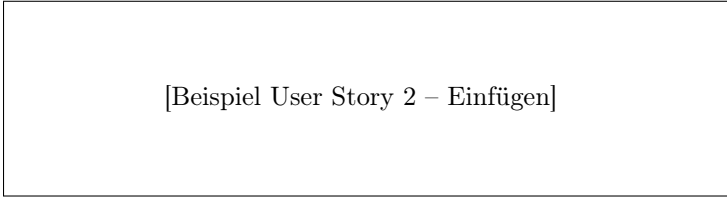
[Sprintübersicht Screenshot – Einfügen]

Abbildung 7: Sprintübersicht



[Beispiel User Story 1 – Einfügen]

Abbildung 8: Beispiel User Story 1



[Beispiel User Story 2 – Einfügen]

Abbildung 9: Beispiel User Story 2

Die folgenden zwei User Stories zeigen exemplarisch den Detailgrad und die Schätzung im Team – eine einfache Kernfunktion und eine komplexere Erweiterung.

8.2 Zeit- und Meilensteinplanung

Das Projekt „Yabloko“ erstreckte sich über das zweite Studienjahr. Bereits im dritten Theoriesemester wurden erste Ideen entwickelt und die grundlegenden User Stories erstellt. Die eigentliche Entwicklungsphase begann im vierten Theoriesemester nach der Praxisphase.

Meilenstein	Beschreibung	Zeitraum	Status
M1	Projektstart & Teamorganisation	Nov 2025	Erreicht
M2	User Stories erstellt & priorisiert	Nov – Dez 2025	Erreicht
M3	Entwicklungsumgebung eingerichtet, Aufgaben verteilt	Apr 2026	Erreicht
M4	Grundlegende Spielmechaniken implementiert	Apr – Mai 2026	Erreicht
M5	Gegner, Checkpoints & Power-Ups fertiggestellt	Mai 2026	Erreicht
M6	Menüsystem, HUD & Audio vollständig integriert	Mai – Jun 2026	Erreicht
M7	Tests durchgeführt, CI/CD-Pipeline eingerichtet	Jun 2026	Erreicht
M8	Dokumentation finalisiert & Projekt abgegeben	25. Jun 2026	Erreicht

Tabelle 11: Meilensteinplan

Die Entwicklungsphase erstreckte sich über insgesamt 8–9 Sprints mit wöchentlichen Meetings. Die umfangreiche Anforderungsanalyse im Wintersemester ermöglichte einen strukturierten und effizienten Entwicklungsstart im Sommersemester.

8.3 Rollenverteilung im Team

- **Michael – Game Logic Developer:** Verantwortlich für die Implementierung zentraler Spielmechaniken, insbesondere der Bewegungssteuerung, Kollisionsverarbeitung und weiterer Kernfunktionen.
- **Schayan – Level & Environment Designer:** Verantwortlich für die Gestaltung der Level, die Platzierung von Spielobjekten sowie die Entwicklung der Spielumgebung.
- **Bartolmay – UI & Menü Design:** Verantwortlich für die Gestaltung und Implementierung der Benutzeroberflächen, Menüs sowie verschiedener HUD-Elemente.
- **Maksym – Projektleitung, Dokumentation & Leveldesign:** Verantwortlich für die Projektkoordination, die Planung der Entwicklungsaktivitäten sowie die Erstellung und Pflege der Projektdokumentation. Zusätzlich beteiligt an der Gestaltung von Levelabschnitten.
- **Tim – Sound & Game Logic Developer:** Verantwortlich für die Erstellung und Integration von Musik und Soundeffekten sowie Mitwirkung an physikbasierten Spielmechaniken und Gameplay-Anpassungen.

Trotz der klaren Rollenverteilung unterstützten sich die Teammitglieder regelmäßig gegenseitig bei der Umsetzung neuer Funktionen, der Fehlersuche sowie bei Tests und Optimierungen.

8.4 Risikomanagement

Bereits zu Beginn des Projekts wurden verschiedene Risiken identifiziert.

Fehlende Erfahrung: Keines der Teammitglieder hatte zuvor Erfahrungen in der Videospielentwicklung. Dieses Risiko wurde durch Recherche, Tutorials und Wissensaustausch reduziert.

Merge-Konflikte: Durch eigene Branches für jeden Entwickler und einen geschützten `main`-Branch minimiert.

Technische Risiken: Neue Funktionen wurden schrittweise implementiert und regelmäßig getestet, bevor sie in den Hauptzweig übernommen wurden.

Spielbalancing: Bewegungsparameter, Gegnerverhalten und Power-Ups wurden durch regelmäßige Spieltests schrittweise optimiert.

Praxisphase: Durch frühe Anforderungsdefinition konnte die Entwicklung nach der Praxisphase unmittelbar fortgesetzt werden.

9 Evaluation

9.1 Bewertung der Zielerreichung

Das Ziel des Projekts bestand darin, ein funktionsfähiges 2D-Plattformspiel zu entwickeln, das den Anforderungen der User Stories entspricht. Dieses Ziel konnte erfolgreich erreicht werden.

Am Ende des Projekts standen mehrere spielbare Level, verschiedene Gegnertypen, Power-Ups, ein Highscore-System, eine Charakterauswahl sowie ein vollständiges Menüsystem zur Verfügung. Das Spiel wurde während der Entwicklung kontinuierlich in einem spielbaren Zustand gehalten.

9.2 Vergleich mit Anforderungen

Ein Großteil der geplanten Anforderungen konnte erfolgreich umgesetzt werden. Insgesamt wurden zu Beginn rund 100 User Stories erstellt; im Verlauf kamen etwa 10 bis 15 weitere hinzu. Der überwiegende Teil dieser Anforderungen wurde erfolgreich umgesetzt.

Einige ursprünglich geplante Anforderungen konnten innerhalb des verfügbaren Zeitraums nicht vollständig umgesetzt werden. Hierzu zählen insbesondere die vollständige Charakterauswahl mit allen geplanten Charakteren sowie zusätzliche Levelinhalte und erweiterte Audioeffekte. Diese wurden in der MoSCoW-Priorisierung als Kann-Anforderungen eingestuft und stellen somit mögliche Weiterentwicklungen dar.

9.3 Diskussion der Ergebnisse

Die ursprünglich definierten Kernanforderungen konnten erfolgreich umgesetzt werden. Die umfangreiche Planungsphase erwies sich als sinnvoll – durch die frühe Klärung von Anforderungen konnten während der Implementierungsphase deutlich weniger grundlegende Entscheidungen getroffen werden.

Neben den internen Tests innerhalb des Projektteams wurde das Spiel auch von Freunden sowie anderen Studierenden ausprobiert. Dadurch konnten zusätzliche Rückmeldungen zur Steuerung, zum Schwierigkeitsgrad einzelner Levelabschnitte und zur allgemeinen Benutzerfreundlichkeit gesammelt werden. Dabei handelte es sich jedoch nicht um eine systematische Nutzerteststudie. Dennoch halfen diese externen Rückmeldungen dabei, Fehler zu erkennen und einzelne Spielmechaniken weiter zu verbessern.

Gleichzeitig zeigte sich, dass der ursprünglich geplante Funktionsumfang teilweise sehr umfangreich war. Dies verdeutlicht die Bedeutung einer realistischen Aufwandsschätzung bereits in der Planungsphase.

10 Fazit und Ausblick

10.1 Zusammenfassung der Ergebnisse

Im Rahmen dieses Projekts wurde erfolgreich ein funktionsfähiges 2D-Plattformspiel mit der Godot Engine entwickelt. Die Anwendung umfasst die wesentlichen Bestandteile eines klassischen Jump-and-Run-Spiels: steuerbare Spielfigur, verschiedene Gegner, mehrere Power-Ups, Checkpoints, Punktesystem, Highscore-System sowie vollständiges Menüsystem.

Ein besonderer Schwerpunkt lag auf dem vollständigen Softwareentwicklungsprozess – von der Anforderungsanalyse über die Implementierung bis zur Qualitätssicherung. Mit über 100 User Stories, einer CI/CD-Pipeline und 50 dokumentierten Testfällen wurde dieser Prozess konsequent umgesetzt.

10.2 Kritische Reflexion (Lessons Learned)

Eine der größten Herausforderungen bestand in der Teamzusammenarbeit. Technische Probleme waren häufig weniger kritisch als Missverständnisse bei Anforderungen oder Schnittstellen. Regelmäßige Meetings und Discord-Kommunikation erwiesen sich als wichtige Erfolgsfaktoren.

Die Implementierung der Spielphysik erforderte zahlreiche Anpassungen und Tests. Die umfangreiche Planungsphase mit über 100 User Stories führte zu einem höheren Anfangsaufwand, reduzierte jedoch spätere Unsicherheiten erheblich.

Darüber hinaus ermöglichte das Projekt einen umfassenden Einblick in verschiedene Methoden des Software Engineerings: Anforderungsmanagement, Modellierung, Testplanung, Projektmanagement und Versionsverwaltung.

10.3 Weiterentwicklungsmöglichkeiten

- **Bosskämpfe:** Einführung am Ende einzelner Level für zusätzliche Herausforderungen.
- **Mobile Version:** Aufgrund der plattformübergreifenden Möglichkeiten der Godot Engine realistisch.
- **Online-Highscore:** Vergleich der Ergebnisse mit anderen Spielern weltweit.
- **Erweitertes Shop-System:** Zusätzliche Charaktere und neue freischaltbare Inhalte.
- **Mehrspieler-Modus:** Kooperatives oder kompetitives Spielen.

Insgesamt bietet die entwickelte Software eine solide Grundlage für zukünftige Erweiterungen.

11 Anhang

11.1 Wichtige Links

Taiga-Board für User Stories und Sprintplanung:

<https://tree.taiga.io/project/maksmykha-mario/timeline>

GitHub Repository als Ablage für Code:

<https://github.com/kurzmicahel02-hue/RunnerGame>

GitHub Actions (CI/CD Testergebnisse):

<https://github.com/kurzmicahel02-hue/RunnerGame/actions>